CMPE 256 – Project Report: Comparing Linear, Neural, and Graph-based Recommender Models on Implicit Feedback Data

Itzel Xu
San Jose State University
Email: itzel.xu@sjsu.edu

Jose Velasco
San Jose State University
Email: jose.velasco@sjsu.edu

Erik Bai
San Jose State University
Email: erik.bai@sjsu.edu

I. Introduction

A. Problem Statement

Recommender systems are essential for filtering large catalogs of items and providing personalized suggestions on digital platforms. In this project, we address the problem of collaborative filtering with implicit feedback. Unlike explicit rating data, our dataset contains only binary interactions, which signal positive preference but not explicit dislikes. The task is to rank over 38,000 items to predict a personalized top-20 list for each user, with performance measured by NDCG@20.

B. Motivation

Recommender systems have evolved rapidly over the last couple of decades, with many models claiming state-of-the-art performance. However, the most advanced or complex model is not always the best fit for a given dataset. Our project is motivated by the need to evaluate and compare four different recommendation architectures and identify the best approach for a sparse implicit-feedback dataset:

- **EASE** – An efficient, high-performing linear model that uses shallow autoencoders. Despite its simplicity, it often outperforms complex deep learning models on top-N recommendation tasks [1].
- **NCF** – A model that replaces the dot product in matrix factorization with an MLP, enabling non-linear user–item interactions.
- **LightGCN** – A graph-based model that propagates embeddings over the user–item bipartite graph to capture high-order connectivity, simplifying standard GCNs by removing feature transformations and non-linearities.
- **GraphGPS** – A graph transformer architecture that combines a GNN with a transformer encoder and positional encodings to incorporate both local and global signals in the graph [2].

C. Objectives

The primary objective is to conduct a comprehensive comparative analysis of these four architectures. Specifically, we aim to:

- Develop and train four models on the provided implicit-feedback dataset.
- Evaluate each model using NDCG@20 to assess ranking quality.
- Investigate the trade-off between model complexity and performance.
- Select the best-performing model for this dataset.

D. Project Repository

The full implementation, including code for data preprocessing, model training, evaluation, and hyperparameter search, is available at: GitHub: https://github.com/Jose-Velasco/multi-model-recommender.

II. System Design and Implementation Details

A. Algorithms Considered

1) EASE (Embarrassingly Shallow AutoEncoder for Sparse Data):

a) Model Summary: EASE is a linear collaborative-filtering model that learns an item–item weight matrix B via a ridge-regularized closed-form solution [1]. Let X be the user–item interaction matrix and λ a regularization parameter. The Gram matrix and its inverse are given by:

$$G = X^\top X + \lambda I, \tag{1}$$

$$P = G^{-1}. \tag{2}$$

The weight matrix B is computed as

$$B_{ij} = \begin{cases} -\frac{P_{ij}}{P_{jj}}, & i \neq j, \\ 0, & i = j, \end{cases} \tag{3}$$

so that the diagonal is zero. The model does not require negative sampling or iterative optimization; training is entirely closed-form.

b) Why Included: EASE is known to perform well on implicit feedback and top-N ranking tasks with relatively little tuning. It serves as a robust and interpretable baseline against which we compare BPR-MF, NCF, LightGCN, and GraphGPS.

c) Complexity: The main computational challenge is the inversion of the $|I| \times |I|$ Gram matrix, which has time complexity $\mathcal{O}(|I|^3)$ and memory complexity $\mathcal{O}(|I|^2)$. For our dataset, there are 31,668 users and 38,048 items, producing a Gram matrix with roughly 1.45×10^9 entries (about 11 GB in float64). By truncating to float32 (about 5.8 GB) and using a

PyTorch-based pipeline, EASE remains memory-intensive but feasible on a modern machine.

Our PyTorch EASE pipeline is adapted from Steck's original formulation [1] and the TorchEASE open-source implementation by Franck [5]. For comparison, we also experimented with a SciPy/NumPy "vanilla" EASE implementation based on an online tutorial [6].

2) NCF (Neural Collaborative Filtering): To overcome the limitations of linear models that are unable to capture complex, non linear relationships, we select Neural Collaborative Filtering. NCF is a simple and lightweight model and can serve as a baseline. NCF models non-linear user interactions by combining generalized matrix factorization (GMF) and an MLP in a two-tower fusion. NCF's inclusion of an MLP allows it to capture complex preference patterns. The simplicity of NCF allows for smaller parameters; therefore, it is faster to train. Furthermore, NCF will be used to compare linear methods, such as EASE and graph-based methods. This will allow us to evaluate how non-linearity helps in a sparse implicit setting.

3) LightGCN: To address the limitation that EASE models user–item interactions independently, we adopt LightGCN, which explicitly models high-order connectivity over the user–item bipartite graph [3]. Classical GCNs inherit feature transformations and non-linear activations from node-classification tasks, which may be redundant for collaborative filtering. LightGCN simplifies this by removing these components, retaining only linear neighborhood aggregation in the embedding space.

4) GraphGPS: The Graph Transformer (GT) GraphGPS was considered for its ability to incorporate long-range dependencies, leverage user-item relational signals, and experiment with a larger model. First, GraphGPS is a hybrid model that integrates a Graph Neural Network (GNN) layer and a transformer encoder block in parallel [2]. This fusion allows the model to leverage a GNN's local view, with the global view that a transformer block enables. These two components alleviate known GNN limitations, for example, oversmoothing, overquashing, and long-range dependencies. This means that GTs capture important signals that GNNs would struggle to model. In particular, when multiple message passing operations are performed, all nodes will converge to the same embedding, thus all valuable patterns and signals are lost. Second, in the context of recommendation systems, this means that GTs can not only recommend a user popular items that similar users like, but can potentially recommend items that are not popular. The transformer component enables node-to-node interactions regardless of hop distance and connectivity; therefore, items with little to no edges can still be considered. While GTs can incorporate long-range dependencies, in a sparse network, this extra information may act as noise, leading to a decrease in performance. Additionally, the quadratic nature of the transformer block can make GTs infeasible. Overall, GraphGPS was selected as it can capture global graph signals, complex local and global collaborative signals, and test how larger models perform on this dataset.

B. Technologies and Tools Used

1) EASE: We use pandas and NumPy for data loading and preprocessing, and PyTorch for linear algebra operations. Hyperparameters are tuned via a deterministic grid search over λ, with NDCG@20 recorded on a fixed validation split. The final model uses the closed-form EASE solution with `torch.linalg.inv` applied to a dense $|I| \times |I|$ Gram matrix, with optional user-row normalization and IDF weighting.

An initial version implemented with SciPy and NumPy using sparse CSR matrices encountered numerical issues and memory pressure for the float64 Gram matrix. Even after switching to float32, near-zero diagonals in P and runtime warnings such as "invalid value encountered in divide" led to nearly identical recommendation lists across users. The PyTorch implementation showed improved stability but remained sensitive to λ and memory constraints.

2) NCF: This implementation utilized TensorFlow and Keras to build the NCF models. Keras provided the flexibility to construct dual-pathway architectures. For data processing, NumPy and Pandas were employed for loading and parsing the raw data, while Numpy was used for array manipulation like negative sampling. To monitor the model performance, Matplotlib and Tqdm are also used to plot the training loss and validation metrics over epochs and provide real-time progress bars during data loading and generating recommendations.

3) LightGCN: The implementation utilizes TensorFlow for the computation graph and automatic differentiation. We employed a custom training loop using tf.GRadientTape rather than the standard .fit method. This allowed us to explicitly control the negative sampling process and the batch-wise BPR loss calculation, ensuring that we could efficiently handle the large number of negative samples required for convergence.

4) GraphGPS: Our GraphGPS implementation leveraged PyTorch Geometric (PyG), Networkx, Ray Tune, and MLflow. First, PyG was used as it is primarily a GNN library with useful link prediction utilities and graph mini-batch sampling. For example, the classes `RandomLinkSplit`, `NegativeSampling`, and `LinkNeighborLoader` were vital in speeding up development; they had methods that easily split into train, validation, and test sets based on the link. Also, PyG provides building blocks for creating custom GNN and GT models; for instance, GPSConv can be stacked to build deeper GraphGPS models. To make training feasible, `LinkNeighbrLoader` allows the creation of mini-batches based on links and incorporates node sampling strategies. Second, Networkx was used to visualize the user-item bipartite graph structure and to mine node-level features. For instance, PageRank, betweenness, degree, and constraint were calculated and transformed into node features. Next, Ray Tune was utilized to perform a hyperparameter search for the GraphGPS model, as it integrates easily with PyG. Lastly, MLflow was used to log and monitor training and hyperparameter search.

C. System Architecture

1) EASE: The EASE component follows the architecture of Steck [1]. It can be viewed as a linear autoencoder without

hidden layers that learns a dense item–item weight matrix ss shown in Fig. 1. Instead of learning low-dimensional user and item embeddings, EASE learns B and predicts scores as:

$$\hat{y}_u = x_u B, \tag{4}$$

where x_u is the sparse binary indicator vector of items interacted with by user u. The diagonal of B is zero so that each item's score depends only on other items. EASE is integrated into our recommender pipeline as a baseline model. Further implementation details are provided in Section II-D1.

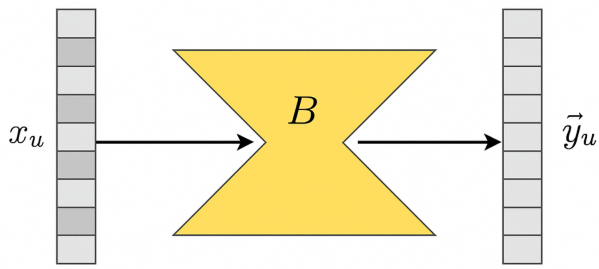

Fig. 1. EASE architecture (adapted from [1])

2) NCF: Our NCF implementation follows the NeuMF architecture in the orignal paper [4], which combines a Generalized Matrix Factorization (GMF) pathway with a Multi-Layer Perceptron (MLP). User and item IDs are first mapped from sparse one-hot vectors into dense embeddings, with GMF and MLP each using their own embedding tables to allow different representational capacities. The model then splits into two branches: GMF computes an element-wise product of the user and item embeddings to capture linear relationships, while MLP concatenates the embeddings and feeds them through several ReLU layers to model non-linear interactions. The outputs of both branches are concatenated and passed to a final sigmoid layer trained with binary cross-entropy, producing the predicted probability that a user will interact with a given item.

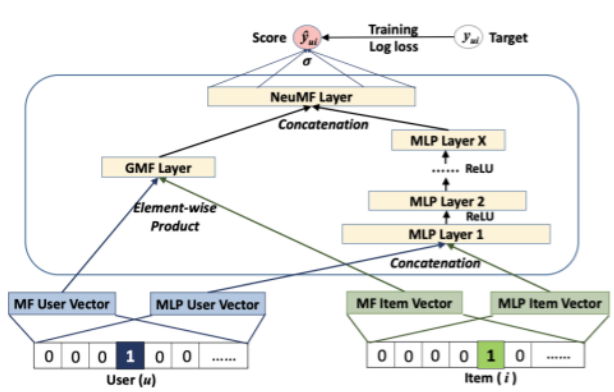

Fig. 2. Neural Matrix Facturization (NeuMF) architecture fusing GMF and MLP (adopt form [4])

3) LightGCN: Our implementation follows the architecture shown in Figure 3, with data flowing through three main components. First, we initialize trainable embedding lookup tables for both users and items, treating the embedding dimension as a tunable hyperparameter explored between 32 and 64. Next, the model applies Light Graph Convolution (LGC), which propagates embeddings through the user–item bipartite graph using the update rule $e^{(k+1)} = \tilde{A}e^{(k)}$, where A is the symmetrically normalized adjacency matrix. This propagation smooths the latent representations so that connected users and items move closer in embedding space. To avoid over-smoothing, the system does not rely solely on the deepest layer; instead, it aggregates embeddings from all layers 0 through K using mean pooling to form the final representations. Recommendations are then produced by computing the inner product between the aggregated user and item embeddings.

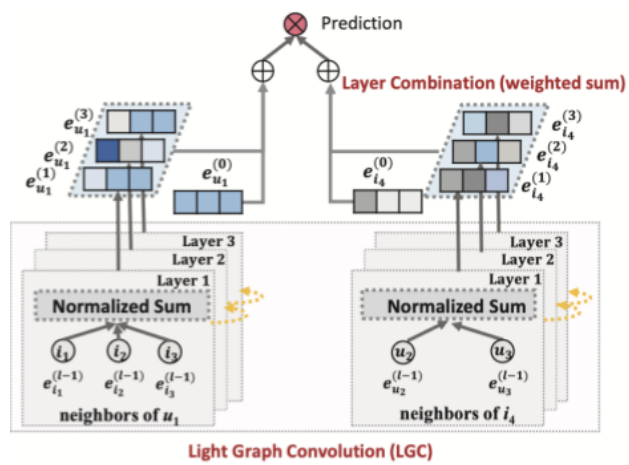

Fig. 3. LightGCN Architecture (adapted from [3].)

4) GraphGPS: Our GraphGPS model consists of 4 key components: a GNN, a transformer encoder, positional encodings, and a multi-layer perception (MLP) scoring head. The GNN component utilizes GINE as its message passing backbone to incorporate topological and user-item collaborative signals. GINE was utilized for its highly expressive and learnable node aggregation function during message passing. Next, GPSConv already has the transformer encoder built in, but still needs a key piece, positional encodings. Because the transformer performs node-node attention regardless of edge connectivity, we inject a node's relative and global graph topological structure into its embeddings. Both the Transformer and GNN operations are performed in parallel, and their outputs are summed and fed into a 2-layer MLP with skip connections and batch norm to produce the next layer's hidden embeddings. GraphGPS's architecture is illustrated in the following figure. The key innovation is the parallel computation of the GNN and Transformer components, as this allows each component to extract valuable features and signals independently [2]. Then, their outputs are summed with skip connections and finally projected and mixed using a 2-layer MLP.

The final GraphGPS layer embeddings for both users and items are concatenated to produce pairwise positive and negative candidate edges. Thus, during training, each user has a set of candidate edges that includes a single positive edge and negative edges $\{e_{p0}, e_1, \ldots, e_n\}$, where e_p and e_n are positive edges and negative edges, respectively. These pairwise user-item embeddings are fed into the 2-layer MLP head to produce scores for each pair.

$$\text{MLP} = \text{Dropout}\big(\text{GELU}(W[X^u_{GPS} \| X^i_{GPS}])\big);$$

$$\text{MLP}_{\text{head}} = \text{MLP}\big(\text{MLP}(X^u_{GPS} \| X^i_{GPS})\big)$$

Where X^u_{GPS} and X^i_{GPS} refer to the user and item of the last GraphGPS layer's output, respectively; $\|$ is concatenation and MLP_{head} outputs the final scores for ranking. These scores act as the strength of a user interacting with an item, with lower values indicating a user is less likely to interact with the item and higher values indicating a user is more likely to interact with it. Therefore, a ranking can be derived from these scores to produce recommendations. Finally, during recommendation, these embeddings are cached to speed up top-k recommendations for each user or other downstream tasks.

D. Data Flow and Workflow

1) EASE: The EASE tuning pipeline we implemented consists of the following steps:

1) **Parsing and k-core filtering:** We read the file train-1.txt line by line. Each line is transformed into pairs $(\text{user}, \text{item})$. Next, we implement a 3-core filter on the items (`item_min` $= 3$), which means that any item we keep must be associated with at least three users. Since all 31,668 users already meet the minimum user-frequency requirement, we maintain `user_min` $= 1$ and do not remove any users.
2) **Warm stratified split for tuning:** To tune the hyperparameters, we randomly divide each user's items into a training set and a separate evaluation set. The items are shuffled using a fixed random seed (42), with around 80% allocated to training and 20% to evaluation. We make sure that each user retains at least one item in both the training and evaluation sets whenever possible. Additionally, a *warm-eval* constraint is applied, ensuring that evaluation items are represented by at least two different users globally, so that the model has prior knowledge of every evaluation target.
3) **Weighting and sparse tensor construction:** After the split, external user and item IDs are mapped to contiguous internal indices $0, \ldots, |U| - 1$ and $0, \ldots, |I| - 1$ via dictionaries `user2idx` and `item2idx`. For each training interaction (u, i) we compute a weight

$$w_{u,i} = \begin{cases} \dfrac{1}{\sqrt{|I_u|}} \cdot \text{IDF}(i), & \text{if user-row norm} \\ & \text{and IDF are enabled} \\ 1, & \text{otherwise} \end{cases}$$

where $|I_u|$ is the number of items for user u and $\text{IDF}(i)$ is an inverse document-frequency term based on item popularity. These weights become the values of a PyTorch sparse COO tensor $X \in \mathbb{R}^{|U| \times |I|}$ constructed with `torch.sparse_coo_tensor`.
4) **Closed-form training and recommendation:** We compute the dense Gram matrix

$$G = X^\top X$$

on CPU, add L2 regularization $G \leftarrow G + \lambda I$, and invert it with `torch.linalg.inv` in float32 to obtain $P = G^{-1}$. The item–item weight matrix is then

$$B = -\frac{P}{\text{diag}(P)},$$

with the diagonal of B set to zero. For each user u, we consider their interaction row xu as a sparse indicator vector, calculate scores $\hat{y}_u = x_u B$, mask all items that have already been seen during training by giving them a large negative score, and then select the top 20 remaining items as recommendations. For the final leaderboard submission, we retrain EASE on the complete filtered dataset (without any splits) using the best λ and weighting configuration, and provide one line per user containing 20 item IDs (excluding user IDs) in the same order as `train-1.txt`.

- **Heavy-user handling and IDF weighting:** We cap each user to at most 300 training interactions (`cap_per_user` $= 300$) so that very active users do not dominate the SGD updates. Positive interactions are optionally reweighted with item IDF to down-weight extremely popular items.
- **Triplet generation:** During each epoch, we iterate over the capped training pairs (u, i) and, for each, uniformly sample one negative item j that user u has not interacted with (`n_neg` $= 1$). These triplets (u, i, j) are used to compute the BPR loss and update user/item embeddings with SGD.
- **Training and validation:** We train for 30 epochs with learning rate `lr` $= 2 \times 10^{-3}$ and L2 regularization $\lambda = 10^{-4}$ (or other values from our grid). The average BPR loss is logged every epoch, and NDCG@20 on the held-out split is evaluated every five epochs to track generalization.
- **Recommendation:** At inference time, scores $s(u, i)$ are computed for all items, previously seen items are masked, and the top-20 unseen items per user are output as recommendations.

2) NCF: Our implementation integrates two complementary sub-architecture: Generalized Matrix Factorization (GMF) and Multi-Layer Perceptron (MLP) into a unified NeuMF framework. The model uses a global embedding dimension of 32, split evenly so that the GMF and MLP pathways each receive 16-dimensional user and item embeddings. The GMF component captures linear user–item relationships through an element-wise product of its embeddings, while the MLP

component models non-linear interactions by concatenating its embeddings and passing them through a three-layer tower with 64, 32, and 16 units, applying a 0.2 dropout rate after the first two layers to reduce overfitting. The outputs from both pathways are then concatenated and passed into a final Sigmoid-activated dense layer to generate a probabilistic interaction score. For training, we adopt a static negative sampling strategy in which four unobserved items are sampled for every positive interaction, producing a balanced 1:4 positive-to-negative ratio that provides sufficient contrastive signal. The model is optimized using Binary Cross-Entropy loss with the Adam optimizer, and trained for five epochs with a batch size of 256, enabling efficient convergence under this sampling scheme. To ensure that training improvements translate to meaningful ranking performance, we implement a custom NDCGCallback for evaluation. At the end of each epoch, the callback randomly selects 100 validation users and computes NDCG@20, providing a ranking-oriented metric that aligns with the recommendation task and complements the BCE loss used during optimization.

3) LightGCN: We implemented the system using TensorFlow to leverage its efficient sparse tensor operations. The implementation required specific design choices to handle the dataset's sparsity. First, we construct the sparse adjacency matrix. The raw dataset contains approximately 1.2 million interactions. Representing this as a dense matrix (31k X 38k) would require prohibitive memory. We utilized scipy.sparse to construct the adjacency matrix in Coordinate format and normalized it symmetrically on the CPU. This matrix is then converted to a TensorFlow SparseTensor, allowing us to perform graph convolution using `tf.sparse.sparse_dense_matmul`, which scales linearly with the number of interactions rather than the number of users. Next, instead of the standard .fit() method in Keras, we implement a custom training loop using `tf.GradientTape`. So, we can implement the Bayesian Personalized Ranking (BPR) Loss, which requires sampling specific triplets (u, i, j) where u is a user, i is a positive item, and j is a negative item within each batch. The loss function is defined as

$$L_{\text{BPR}} = -\sum \ln \sigma(\hat{y}_{ui} - \hat{y}_{uj}) + \lambda \|\Theta\|^2$$

This loss strictly optimized the ranking order, pushing the scores of observed interactions higher than unobserved ones. Finally, we utilized the Adam optimizer with a default learning rate of 0.001 and batch size of 8192 to speed up training on the GPU. L2 regularization was applied to the embeddings to prevent overfitting.

4) GraphGPS: At a high level, our GraphGPS model pipeline transforms the interaction text data into a graph enriched with node centralities to train the model. First, we construct an undirected bipartite graph of user and item nodes. This is done by processing the training text data to generate user and item nodes, where edges indicate that a user interacted with an item. Then, node centralities, positional encodings, and relative edge features are computed. In brief, we derived multiple graph transformation classes that would compute these manual node features and incorporate them into the graph during graph construction. These manual features incorporate valuable importance and topological information about a user's position in the graph. Next, subgraphs are constructed by splitting the whole graph into training, validation, and test sets. To make training feasible for each node, we sample a subset of neighbors at each hop. Finally, for negative sampling, each positive edge is paired with a set of negative edges that a user has not interacted with. The set of negative edges that a user has not interacted with is precomputed to speed up candidate edge generation. It is important to note that when splitting edges, two sets are constructed: message passing edges and supervision edges. To explain, message passing edges, as the name suggests, are for performing message passing in the GNN component. In contrast, supervision edges are inputs to the BPR (Bayesian Personalized Ranking) loss to perform backpropagation. In other words, the supervision edges are positive user-item edges

III. Experiments / Evaluation

A. Dataset Overview

The dataset `train-1.txt` contains 31,668 users, 38,048 items, and 1,237,259 interactions, resulting in a density of approximately 0.10%. Interaction counts are highly skewed Fig. 4 5: the top five items have between 1,014 and 1,258 interactions. Users have at least 16 interactions, with a subset of highly active users having between 900 and 1,848 interactions.

B. Experimental Methodology

1) EASE: For EASE we reuse the parsing, k-core item filter, and warm per-user stratified split described in Section II-D1. In this context, EASE views all training interactions as positive feedback and refines the closed-form solution without using any sampled negatives.

We start by conducting a grid search for the regularization parameter λ on the warm evaluation split. Specifically, we explore values of

$$\lambda \in \{100, 200, 300, 400, 500, 600, 700, 1000, 2000\},$$

with user-row normalization enabled and IDF weighting turned off. Figure 6 illustrates NDCG@20 as a function of λ on a logarithmic scale. The performance rises from an NDCG@20 of approximately NDCG@20 ≈ 0.0371 at $\lambda = 100$, reaching a peak of around 0.0404 at$\lambda = 500$, before gradually declining as the regularization becomes overly strong. Consequently, we set $\lambda = 500$ for the following ablation studies.

Next, we assess the impact of IDF reweighting while maintaining $\lambda = 500$ and user-row normalization activated. Figure 7 contrasts two setups: one without IDF weighting (`IDF_weight = False`) and the other with IDF weighting (`IDF_weight = True`). The validation NDCG@20 improves from 0.0394 to 0.0517, representing a relative increase of about 31%. This indicates that reducing the weight of extremely popular items and slightly elevating the importance

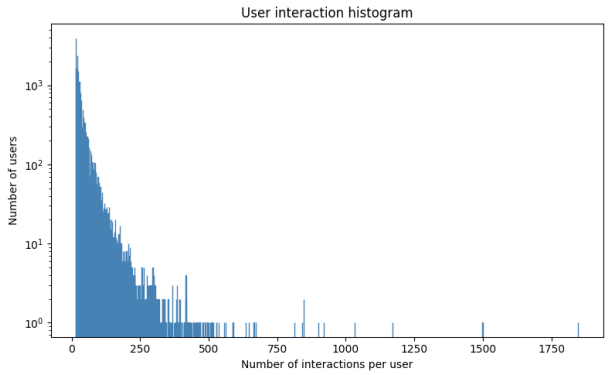

Fig. 4. User interaction histogram

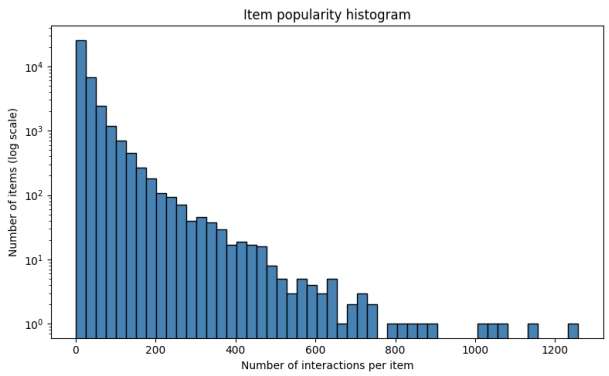

Fig. 5. Item popularity histogram

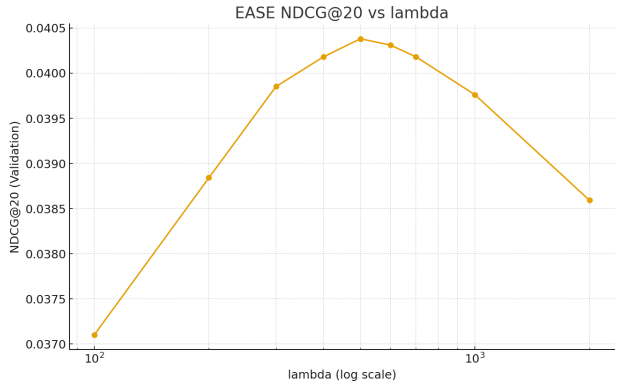

Fig. 6. Validation NDCG@20 for EASE as a function of the regularization parameter λ (log scale).

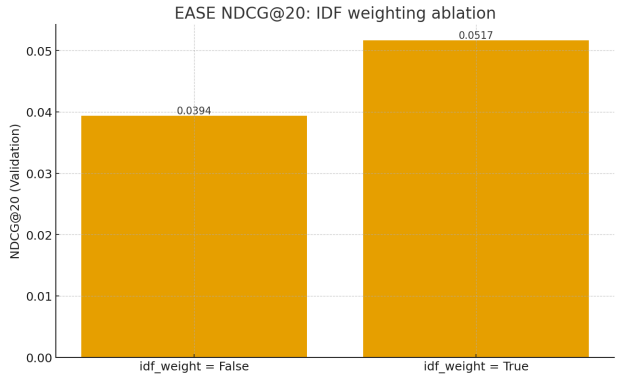

Fig. 7. EASE validation NDCG@20 with and without IDF weighting at $\lambda = 500$.

of less common items enables the linear EASE model to better identify user-specific preferences within this dataset.

Based on these experiments, our final EASE configuration uses user-row normalization, IDF weighting, and $\lambda = 500$. This tuned model achieves validation NDCG@20 ≈ 0.0517 and serves as our strongest linear baseline before comparing to neural and graph-based models.

2) NCF: Our preprocessing pipeline began by parsing the dataset into a dictionary of user–item interactions, directly mapping the integer user and item IDs to embedding indices. The maximum user ID (N) and item ID (M) determined the dimensionality of the embedding layers, ensuring that all entities were represented. For training, we adopted a 90/10 hold-out validation strategy by using validation split in `model.fit()`. This reserved 10% of the training data allowed us to monitor binary cross-entropy loss and accuracy on the validation subset, helping to track convergence and mitigate overfitting. Beyond standard classification metrics, we implemented a ranking-oriented NDCG evaluation protocol executed at the end of each epoch. For efficiency, 100 validation users were randomly sampled, and a leave-one-out strategy was applied where each user contributed one positive item and 99 negative items to form a 100-item candidate set. The model scored these items, and NDCG@20 was computed based on the rank of the single positive item, providing a lightweight estimate of recommendation quality. For the leaderboard submission, final predictions were generated by scoring all approximately 38k items for each user. The top 20 highest-scoring items were selected while explicitly masking out any items the user had previously interacted with during training, ensuring that recommendations were novel and meaningful. To isolate the contribution of the hybrid architecture itself, we adopted a staged experimental workflow. We first implemented and evaluated a standalone MLP model to establish a purely non-linear neural baseline without any matrix-factorization component. After verifying stable convergence and baseline performance, we advanced to the full NeuMF fusion architecture. This iterative approach allowed us to rigorously assess whether combining the linear interaction modeling of GMF with the non-linear expressiveness of MLP yielded measurable improvements in ranking performance compared to the pure neural baseline.

3) LightGCN: Since our dataset is very sparse and the ground truth for the leaderboard test set is hidden, we split the dataset into a random 80% training set and 20% validation set. The validation set used to evaluate the performance of each combination of hyperparameters during the grid search. For the final output, we retrained the model with optimal hyperparameters on the full available data to unlock the full potential of the model.

The dataset only has implicit positive interactions, the model

TABLE I
LIGHTGCN HYPERPARAMETER SEARCH SPACE.

Category	Key Hyperparameters
Learning Rate	{0.005, 0.01, 0.015}
Embedding Dimension	{32, 64}
Number of Layers	{2, 3}

TABLE II
CONDENSED HYPERPARAMETER SEARCH SPACE (SEE APPENDIX A FOR FULL RANGES AND NESTED CONFIGURATIONS).

Category	Hyperparameters (Condensed)
Training	Batch size: 128–1024; LR: {1e-2, 1e-3, 1e-4}; Weight decay: 1e-2–1e-5; BPR λ: 0.1–0.5; Neg. sampling ratio: 2–5
Optimization	ReduceLROnPlateau (factor = 0.5, patience = 3, min LR = 1e-5)
Graph Sampling	Neighborhood size per layer $\in [3, 10]$
GraphGPS Model	Channels: {128, 256, 512}; Layers: 1–4; Attention heads: {2, 4}; Dropout: 0.1–0.5; Activation: {ReLU, GELU}
MPNN Component	Layers: 1–4; Dropout: 0.1–0.5; MLP width aligned to embedding size
Scoring Head	Hidden dim: 512–4096; Dropout: 0.1–0.5

must learn to distinguish between observed interactions and unobserved ones. The LightGCN paper demonstrated that simple uniform sampling is both computationally efficient and highly effective for optimizing BPR Loss in LightGCN [3]. We employed a uniform random negative sampling strategy with 1:1 ratio to generate training triplets (u, i, j), where i is positive interaction and j is negative interaction that uniformly randomly selects from the set of non-positive items.

We performed a grid search over the key hyperparameters that control model complexity and convergence. The search space included the following:

The best model was selected based on the highest validation NDCG@20 over 50 epochs. This approach will maximize ranking performance rather than just minimizing BPR loss.

4) GraphGPS: An extensive hyperparameter search space was set up using Ray Tune across models, training, and optimization parameters Table II. An 80%, 10% and 10% edge split was performed to produce train, validation, and test sets, respectively. Furthermore, in each set, 60% of the edges were for message passing, and the remaining were for supervision. A random negative sampling ratio controlled by Neg. sampling ratio hyperparameter was employed over a user's set of non-interacted items. Additionally, the Normalized Discounted Cumulative Gain (NDCG) metric was used to perform early stopping and for learning rate scheduling. Finally, a final model was retrained on all of the data. For inference, the final GraphGPS embeddings are cached, and for each user, we score them with items they have not interacted with, and we take only the top 20

C. Analysis

1) EASE: EASE was the strongest among the models, Figure 8. This shows that EASE's simple architecture performs

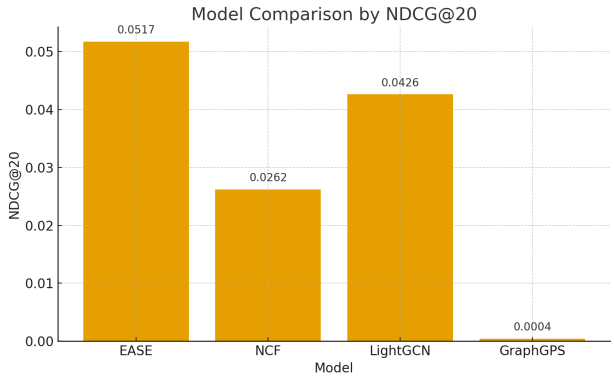

Fig. 8. Model Performance measured by NDCG@20, showing EASE as the top performer.

well on implicit feedback data with high sparsity. EASE's closed-form effectively models global user-item patterns. Additionally, the usage of row-normalization and IDF weighting helps suppress popularity bias. This is significant because items with few interactions can still be recommended, thus alleviating the item cold-start problem.

2) NCF: The NCF experiments revealed a major mismatch between local validation and leaderboard performance, exposing a clear overfitting issue. Although the training plots showed strong local validation results, approximately 0.57 NDCG for MLP and 0.69 for NeuMF. These scores were misleading because the evaluation protocol only ranked the single positive item against 99 randomly sampled negatives. When the models were evaluated on the leaderboard, which ranks each positive item against the full catalog of nearly 38k items, performance dropped sharply to 0.0262 for MLP and 0.0236 for NeuMF. This demonstrated that the models had learned to discriminate positives only within a small and overly simplified negative set, failing to generalize to the true large-scale ranking task. Further analysis showed that the fused NeuMF architecture unexpectedly underperformed the standalone MLP model. Unlike the original NCF results, NeuMF's GMF+MLP fusion degraded performance due to the absence of pre-training for the GMF and MLP branches. Because NeuMF is a highly non-convex model, training it end-to-end from scratch made the GMF pathway unstable, causing it to inject noise rather than complementary linear signal. As a result, the MLP component's stronger non-linear representation was hindered, explaining why plain MLP became the more reliable baseline in our experiments.

3) LGCN: For LightGCN, to evaluate the effects of our optimizations, we compared the training dynamics across two development phases. In the baseline configuration, Figure 3.4.4, which used a batch size of 2048 and standard random initialization, the model converged smoothly but quickly plateaued, with validation NDCG stabilizing around 0.036 on leaderboard. Although training was stable, the model appeared to settle into a shallow local minimum, likely because the smaller batch size and uninformed initialization limited its

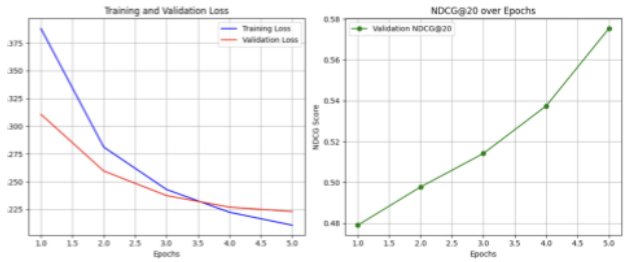

Fig. 9. MLP path only.

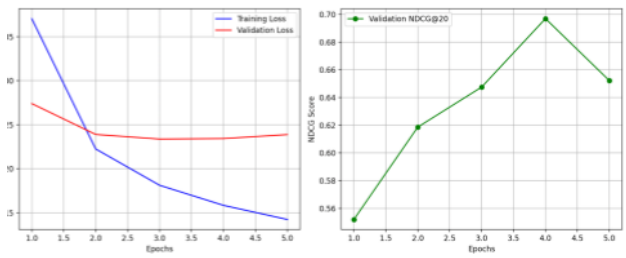

Fig. 10. Fusion (MLP+NeuMF).

ability to propagate gradients effectively through the graph. In contrast, the optimized implementation in Figure 3.4.4 demonstrated a substantially higher performance ceiling. By adopting Xavier initialization and increasing the batch size to 8192, the model sustained meaningful learning for more epochs and ultimately reached an NDCG of 0.0426 on leaderboard. This was an 18% improvement over the initial version of LightGCN. The training loss and validation NDCG exhibited a clear inverse correlation, indicating that the larger batch size and training on more epochs enabled more stable and expressive gradient updates, allowing the model to capture stronger collaborative signals across the full item catalog. This is completely different from what we saw in the NCF model where we ran into overfitting with only training on 5 epochs.

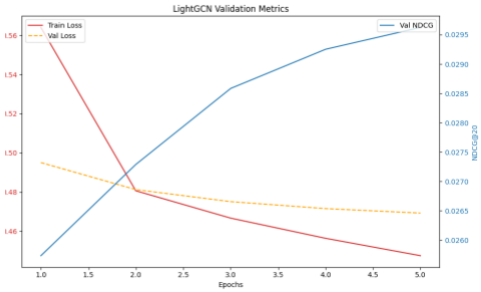

Fig. 11. Early stage of LightGCN.

4) GraphGPS: GraphGPS performed poorly and struggled with training stability, with out-of-memory and low gain. GTs require rich node features, positional encodings, and strong graph modeling of datasets. This dataset is limited to only user-item IDs; thus, the transformer component received few useful signals. Attempts were made to introduce meaningful node features using graph mining techniques such as PageRank,

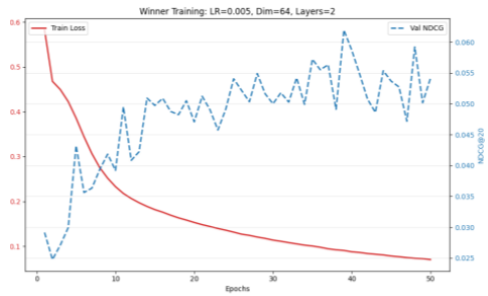

Fig. 12. Final version LightGCN.

degree, and others. Nevertheless, GraphGPS still struggled with these mined node features that inject a node's importance, centrality, and structural topology. GraphGPS has higher modeling capacity, therefore making it easy to overfit the tiny number of positive edges. This can lead to global attention, introducing noise when aggregated with the GNN component. This noise makes training and validation curves extremely unstable; as a result, the model essentially predicts similar scores for all items. This may explain why GraphGPS has a near-zero NDCG@20.

Results indicate that simpler models excel in sparse implicit feedback recommendation. For example, EASE dominated, receiving NDCG@20 = 0.0517. The linear nature of EASE means it is extremely stable and has low variance, leading to consistent recommendations. Furthermore, EASE did not require heavy tuning and can quickly recommend user items. Next, Graph-based models benefit from the graph structure quality. LightGCN utilizes propagation to capture higher-order collaborative signals better than NCF. However, GraphGPS's poor performance may indicate that its complexity and heavy parameterization yield no benefits on sparse implicit feedback. This means that simpler and faster solutions are sufficient. Overall, linear methods not only scale amazingly, but they also outperform more complex models on implicit interaction datasets. At the same time, graph-based models excel only when the data has meaningful graph structure outside of binary interactions.

IV. Discussion and Conclusions

A. Design Decisions

This subsection outlines the main architectural choices and hyperparameter selections made for each model. Refer to Table III.

B. Challenges and Limitations

1) EASE: EASE's main challenge is computational: the model needs to invert a dense $|I| \times |I|$ Gram matrix. With $|I| = 38{,}048$, this matrix contains around 1.45×10^9 entries, which requires a significant amount of memory, even when using float32, making each grid-search run quite costly. Our initial prototype in NumPy/SciPy using float64 quickly ran into memory limitations, and even after switching to float32, we encountered numerical problems such as near-singular

TABLE III
MODEL BEST HYPERPARAMETERS.

Model	Best Params
EASE	$\lambda = 500$, row-norm + IDF
NCF	Sampling ratio = 4; GMF dim = 16; MLP dim = 16; Dropout = 0.2
LightGCN	Sampling ratio = 1; Batch size = 8192; Learning rate = 0.001
GraphGPS	User embedding = 256; Negative sampling ratio = 2; Attention heads = 4

diagonals, divide-by-zero warnings, and nearly identical recommendation lists for different users. This led us to rewrite the training step in PyTorch while keeping everything on the CPU in float32.

Even with the PyTorch implementation, EASE is still quite sensitive to the regularization strength λ: if it's too small and the inverse P becomes ill-conditioned; if it's too large, the model tends to over-smooth item relationships, which can lower NDCG. Additionally, the model is linear and global, meaning the same matrix B is used for all users. This limits its ability to adapt to different user subpopulations or to capture non-linear effects that could be addressed by NCF or GraphGPS. Lastly, despite k-core filtering and the warm-eval constraint, EASE still struggles to personalize for very cold items and users who have only a few interactions left after the split.

2) NCF: A major limitation of our NCF implementation was the use of static negative sampling. To fit the Keras training pipeline, we pre-generated a fixed set of negatives rather than refreshing them each epoch. This likely caused the model to overfit to a narrow set of "easy" negatives and reduced its ability to generalize across the full item space.The model also lacked pre-training for the GMF and MLP branches. NeuMF is difficult to optimize from random initialization, and prior work shows that separately pre-training each tower is key for strong performance. Skipping this step simplified the workflow but likely led to suboptimal convergence. We further had to approximate ranking evaluation due to computational cost. NDCG was calculated on only 100 sampled users per epoch, which kept training efficient but introduced noticeable variance in validation trends. Lastly, the sparsity of the dataset caused the model to plateau quickly, even with dropout applied. This reflects a broader challenge for NCF on implicit-feedback data, where deep models often overfit faster than graph-based alternatives.

3) LightGCN: Throughout the development lifecycle, we encountered several technical challenges that required significant architectural revisions from our original plan. Our early design used the high-level model.fit() API, but this proved too inflexible for dynamic negative sampling, which must be refreshed every epoch in implicit feedback recommendation tasks. The inability to customize the sampling process caused runtime issues and prevented effective learning, leading us to replace the entire training workflow with a fully manual tf.GradientTape loop that allowed precise control over gradient updates and data feeding.

We also faced complications with the loss function. Although LightGCN traditionally uses BPR loss, implementing efficient triplet sampling(u, i, j) in pure Python introduced a severe data-loading bottleneck. Balancing theory and practicality, we switched to a pointwise Binary Cross-Entropy approach with 1:1 negative sampling, which allowed vectorized computation, faster convergence, and competitive ranking accuracy.

Memory constraints also played a key role in shaping the final architecture. Working with an adjacency matrix covering 31,668 users and 38,048 items repeatedly caused notebook kernels to crash due to accidental densification. To address this, we enforced strict sparse-matrix handling: constructing the normalized Laplacian with scipy.sparse and operating on it as a tf.SparseTensor, reducing memory usage from quadratic to linear in relation to the number of interactions.

Finally, switching from BPR to Binary Cross Entropy (BCE) introduced unexpected hyperparameter sensitivity. The commonly recommended learning rate of 0.001 was far too small under BCE, resulting in slow or stalled convergence. Through experimentation reflected in the final tuning range, we determined that higher learning rates between 0.005 and 0.015 consistently produced better training dynamics, illustrating how changes in loss formulation can substantially alter optimization behavior.

4) GraphGPS: A big challenge was working with GraphGPS because of its transformer component. While this implementation utilizes the Performer optimization, it is a heavy model that needs special consideration. For example, node sampling is fundamentally needed as global attention is expensive, but it is one of the reasons for the success of Transformers. Another huge challenge is balancing learned user and item embeddings. In particular, larger learned embeddings lead to an explosion of model parameters, thus slowing down training and inference speed. In general, our implementation of GraphGPS did not work well and performed the worst. Even with regularization such as dropout, weight decay, L2-norm on learned embedding, early stopping, and with sufficient epochs, our GraphGPS performed badly. Future work can consist of improving the current implementation and looking for areas that might be contributing to subpar results. At the same time, introducing global node-node attention on a large implicit interaction graph introduces substantial noise. This means that testing with other forms of positional encodings that work well with sparse implicit interaction is necessary so the transformer block can better discern valuable signals from noise. Lastly, this model's future work can look into other techniques to prevent overfitting, as this could be a reason that our current implementation performed badly.

C. What Worked Well

1) EASE: From a practical perspective, EASE turned out to be the most effective and reliable model in this project once the implementation issues were resolved. After moving to the PyTorch-based pipeline with float32, the closed-form

training became deterministic and easy to reproduce: given the same split and hyperparameters, the model always returns the same B and the same NDCG@20. The small hyperparameter space also made it straightforward to interpret the effects of each design choice. Empirically, the grid search over λ showed a clear but smooth optimum around $\lambda = 500$, with performance degrading only gradually away from this point. This suggests that EASE is not extremely brittle with respect to regularization once we are in a reasonable range. The IDF ablation was particularly successful: enabling IDF weighting improved validation NDCG@20 from 0.0394 to 0.0517, a gain that no other single tweak produced for BPR-MF or LightGCN. Overall, EASE provided the strongest baseline on this dataset and a useful reference point for interpreting the weaker results of the neural and graph-based models.

2) NCF: Several architectural and engineering choices were pivotal to achieving scalable and stable training. Replacing the rigid model.fit() API with a custom tf.GradientTape loop provided granular control over dynamic negative sampling and enabled real-time monitoring of batch-wise loss and epoch-wise NDCG. To handle the large interaction graph efficiently, we implemented graph convolution using `tf.SparseTensor` and `tf.sparse.sparse_dense_matmul` , which allowed the model to scale linearly with the number of interactions and perform message passing on the global graph without materializing a dense adjacency matrix. For the loss function, we adopted Binary Cross Entropy (BCE) with a 1:1 negative sampling ratio instead of the standard BPR loss, streamlining the data pipeline and accelerating training through vectorized operations, ultimately achieving our highest leaderboard score of 0.0426. Additionally, switching from standard random initialization to Glorot Uniform (Xavier) initialization stabilized the model and transformed the training dynamics from to a smoother convergence curve.

3) LightGCN: The NCF experiments revealed that the standalone MLP model served as a surprisingly strong and robust baseline, outperforming the more complex fused NeuMF architecture (0.0262 vs. 0.0236 NDCG). This result shows that, given the sparsity of the dataset, a simpler non-linear model generalized better than an insufficiently optimized hybrid design. In addition, the neural models converged rapidly, reaching peak performance in just five epochs. This fast training cycle made it easy to iterate and quickly diagnose the overfitting behaviors on sparse interaction data.

D. Final Conclusions

- **EASE as the best overall model.** By applying user-row normalization, IDF weighting, and setting $\lambda = 500$, EASE reached the highest NDCG@20 score of (≈ 0.0517) and consistently outperformed all other models on this sparse implicit dataset.
- **Neural models (NCF).** MLP and NeuMF were able to capture certain non-linear effects, but they still fell short compared to EASE and LightGCN. Additionally, they required more tuning and provided only modest improvements over simpler baseline models.
- **Graph-based models.** LightGCN took advantage of the user-item graph and achieved competitive performance (NDCG@20 ≈ 0.0426), while GraphGPS struggled due to its noisy sparse structure, excessive attention requirements, high memory usage, and unstable training.
- **What did not work well.** The implementation of EASE using SciPy/NumPy with a dense float64/float32 inverse proved to be both memory-heavy and numerically unstable, leading to issues like an ill-conditioned Gram matrix, NaNs, and nearly identical recommendation lists. Additionally, GraphGPS failed to convert its expressive capabilities into improved NDCG.
- **Key lessons.** Simple, well-structured linear methods can outperform more complex neural networks and graph transformers when the data is extremely sparse. In these cases, factors like preprocessing, regularization, and weighting—such as k-core filtering, warm per-user splits, row normalization, and IDF—are more significant than simply increasing architectural complexity.

Project Plan / Task Distribution

Actual Contributions

Itzel Xu:

- Data analysis
- Preprocessing (parsed interactions, k-core filtering, and warm 80/20 per-user split)
- Training, grid search and evaluation for EASE and MF-BPR models

Jose Velasco:

- Graph-based visualization
- Full GraphGPS modeling, feature mining, hyperparameter search, training, and evaluation

Erik Bai:

- NCF and LightGCN implementation
- Training optimization, tuning, and evaluation for NCF and LightGCN
- Plot training loss and validation score for analysis

References

[1] H. Steck, "Embarrassingly shallow autoencoders for sparse data," in *Proc. WWW*, San Francisco, CA, USA, May 2019, pp. 3251–3257. doi: 10.1145/3308558.3313710.

[2] L. Rampášek, M. Galkin, V. P. Dwivedi, A. T. Luu, G. Wolf, and D. Beaini, "Recipe for a general, powerful, scalable graph transformer," in *Adv. Neural Inf. Process. Syst. (NeurIPS)*, 2022. [Online]. Available: https://openreview.net/pdf?id=lMMaNf6oxKM

[3] X. He, K. Deng, X. Wang, Y. Li, Y. Zhang, and M. Wang, "LightGCN: Simplifying and Powering Graph Convolution Network for Recommendation," in *Proc. 43rd Int. ACM SIGIR Conf. Research and Development in Information Retrieval*, 2020, pp. 639–648. doi: 10.1145/3397271.3401063.

[4] X. He, L. Liao, H. Zhang, L. Nie, X. Hu, and T.-S. Chua, "Neural Collaborative Filtering," in *Proceedings of the 26th International Conference on World Wide Web (WWW '17)*, Perth, Australia, 2017. doi: 10.1145/3038912.3052569.

[5] J. Franck, TorchEASE: PyTorch implementation of EASE," GitHub repository, 2023. [Online]. Available: https://github.com/franckjay/TorchEASE (Accessed: Nov. 27, 2025).

[6] Step-by-Step Data Science, EASE: Embarrassingly Shallow Autoencoders for Sparse Data," 2023. [Online]. Available: https://www.stepbystepdatascience.com/ease%7D (Accessed: Nov. 27, 2025).

Appendix A
Extra Models

A. BPR-MF (Bayesian Personalized Ranking – Matrix Factorization)

BPR-MF optimizes a pairwise ranking objective for implicit feedback. Instead of predicting ratings, it explicitly optimizes the relative scores of observed interactions versus unobserved ones using triplets (u, i, j), where u is a user, i a positive item, and j a negative item. The loss encourages $s(u, i) > s(u, j)$, making it well-suited to implicit feedback.

Our BPR-MF implementation is a simple matrix factorization model based on NumPy, trained using stochastic gradient descent on the Bayesian Personalized Ranking objective. We utilize pandas to convert the implicit interaction file into (u, i) pairs, then we factor user and item IDs into contiguous indices for dense embedding tables. The user and item latent factors are kept as NumPy arrays and updated directly within a custom Python training loop. For each positive interaction (u, i), we sample one negative item j that user u hasn't interacted with, and we perform the standard BPR update based on the score difference $s(u, i) - s(u, j)$.

We use Matplotlib to log and visualize the average training loss for each epoch and the validation NDCG@20 every five epochs. Additionally, a separate SVD-based analysis script calculates the top singular values of the user-item matrix and their cumulative explained energy to determine a suitable latent dimension. All BPR-MF experiments follow the same preprocessing and evaluation split as EASE to maintain comparability.

The BPR-MF model represents each user u and item i with k-dimensional latent vectors $p_u, q_i \in \mathbb{R}^k$. The predicted relevance score is given by the inner product

$$s(u, i) = p_u^\top q_i. \tag{5}$$

Training optimizes a pairwise ranking loss over triplets (u, i, j), where i is an observed (positive) item and j is a sampled negative item that user u has not interacted with. For each triplet, the BPR loss encourages in Eq. **??**.

The architecture is therefore a standard low-rank MF model without additional layers or side features. At inference time, recommendation for a user u is done by computing $s(u, i)$ for all candidate items and ranking them, masking out items already seen in training. Compared to EASE, BPR-MF trades closed-form training for a stochastic, pairwise optimization that is more directly aligned with ranking.

The BPR-MF pipeline reuses the same parsed interactions, k-core filtering, and warm 80/20 per-user split as EASE (Section II-D1), including the constraint that every evaluation item appears at least once in training. On top of this shared preprocessing, we add a few BPR-specific steps:

BPR-MF is assessed using the same per-user stratified warm split as EASE. To determine an appropriate latent dimension, we initially perform truncated SVD on the user-item interaction matrix and examine the spectrum of singular values along with the cumulative explained energy. We find that about 90% of the energy is accounted for by approximately 200components. Therefore, for the primary BPR-MF experiments, we set the number of factors to $k = 128$, striking a balance between capacity and runtime.

Unless otherwise stated, the default configuration is:

- **Preprocessing:** `user_min` $= 1$, `item_min` $= 3$, warm 80/20 user-wise split, and `cap_per_user` $= 300$ interactions.
- **Model and optimization:** $k = 128$ factors, learning rate `lr` $= 2 \times 10^{-3}$, $\lambda = 10^{-4}$ L2 regularization, `n_neg` $= 1$ negative per positive, `use_idf_weight` $=$ `True`, and 30 SGD epochs.

Figure 13 illustrates that the average BPR loss gradually decreases from around 0.695 to 0.665 over the course of 30 epochs, suggesting that the model is becoming better at favoring positive items over the sampled negatives. However, the validation NDCG@20 remains quite low, only rising from about 3.5×10^{-4} to approximately 7×10^{-4}, which is essentially at the level of random ranking and significantly lower than the EASE baseline (NDCG@20 $\approx 5.17 \times 10^{-2}$). Minor adjustments in λ, k, and the learning rate produced very similar NDCG results, leading us to consider BPR-MF as a weak baseline for this dataset. Therefore, we will concentrate the remainder of our analysis on the more robust EASE and graph-based models.

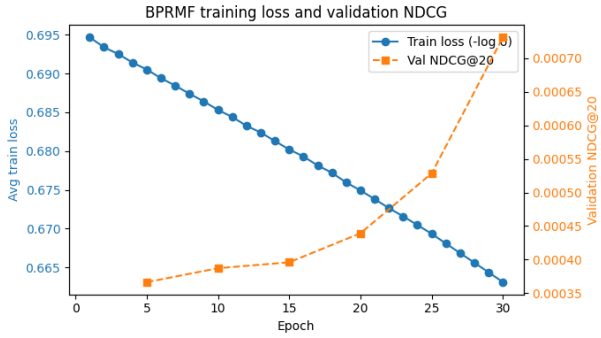

Fig. 13. MF-BPR training loss and validation NDCG@20. Loss decreases steadily, but NDCG@20 remains close to random.

Although it aligns conceptually with implicit-feedback ranking, BPR-MF did not perform well on this dataset. While the training was numerically stable and the BPR loss consistently decreased, the validation NDCG@20 stayed very low. We first ran a grid search over latent dimensions $n_{\text{factors}} \in \{96, 128, 160, 192, 224\}$ and regularization strengths $\lambda \in \{10^{-5}, 5{\times}10^{-5}, 10^{-4}, 5{\times}10^{-4}, 10^{-3}\}$, then narrowed the range to $\lambda \in \{5{\times}10^{-5}, 10^{-4}, 2{\times}10^{-4}, 3{\times}10^{-4}, 5{\times}10^{-4}\}$ based on training-loss curves. Across all combinations, NDCG@20 stayed around 4.3×10^{-4}, essentially at the level of random ranking and far below the EASE baseline (NDCG@20 $\approx 5.17 \times 10^{-2}$).

There are several reasons that could explain this disappointing outcome. Firstly, despite considering up to 224 factors, the basic MF architecture might be too simplistic to effectively capture the complex item-to-item relationships that EASE models represent with a dense weight matrix. Secondly, the pairwise BPR optimization, which uses only one negative sample for each positive, may not provide a strong enough learning signal in a space with over 38,000 items, resulting in many users experiencing little distinction between positive and negative items. Lastly, while we extensively varied the n_{factors} and the regularization parameter λ, we maintained a relatively conservative training schedule (30 epochs with a fixed learning rate) due to runtime constraints. In summary, BPR-MF primarily acts as a weak baseline, highlighting the superior performance of the EASE and LightGCN models on this specific dataset.

APPENDIX B
EXTRA FIGURES, PLOTS AND TABLES

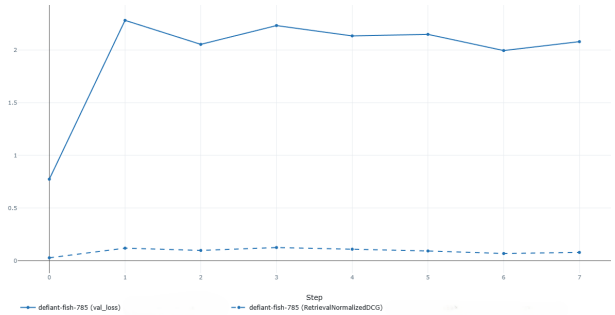

Fig. 14. GraphGPS best model trained on all of the data. solid line is validation loss and dotted line is NDCG.

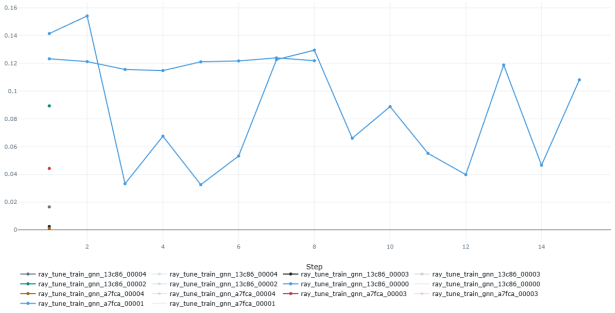

Fig. 15. GraphGPS hyperparameter search NDCG@20 per epoch.

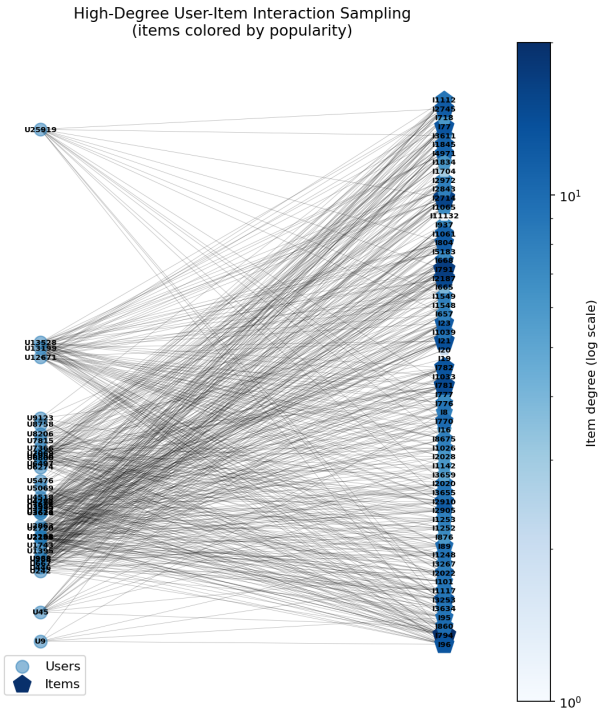

Fig. 16. User-item bipartite graph sampled by taking users and items with the highest degree because visualizing the whole graph is impractical.

TABLE IV
GRAPHGPS TRAINING CONFIGURATION SEARCH SPACE.

Hyperparameter	Values / Description
train_batch_size	128, 256, 512, 1024
optimizer_learning_rate	0.01, 0.001, 0.0001
weight_decay	0.01, 0.001, 0.0001, 0.00001
lr_scheduler.mode	”max”
lr_scheduler.factor	0.5
lr_scheduler.patience	3
lr_scheduler.min_lr	1e-5
neg_sampling_ratio	2, 3, 4, 5
evaluation_num_negative	50, 75
num_neighbors	Per-layer values sampled from [3–10]
BPR_loss_lambda	0.1, 0.2, 0.3, 0.4, 0.5

TABLE V
GRAPHGPS SCORING HEAD CONFIGURATION SEARCH SPACE.

Hyperparameter	Values / Description
input_channels	Same as GPS model channels
hidden_channels	512, 1024, 2048, 4096
dropout	0.1, 0.2, 0.3, 0.4, 0.5

TABLE VI
GRAPHGPS MPNN COMPONENT CONFIGURATION SEARCH SPACE.

Hyperparameter	Values / Description
num_layers	1, 2, 3, 4
mlp_channels	64, 128, 256, 512
dropout	0.1, 0.2, 0.3, 0.4, 0.5

TABLE VII
GRAPHGPS HIGH-LEVEL MODEL CONFIGURATION SEARCH SPACE.

Hyperparameter	**Values / Description**
channels	128, 256, 512
num_layers	1, 2, 3, 4
attn_kwargs.dropout	Random choice from 0.1–0.5
attentions_heads	2, 4
dropout_prob	0.1, 0.2, 0.3, 0.4, 0.5
act_func	”gelu”, ”relu”